

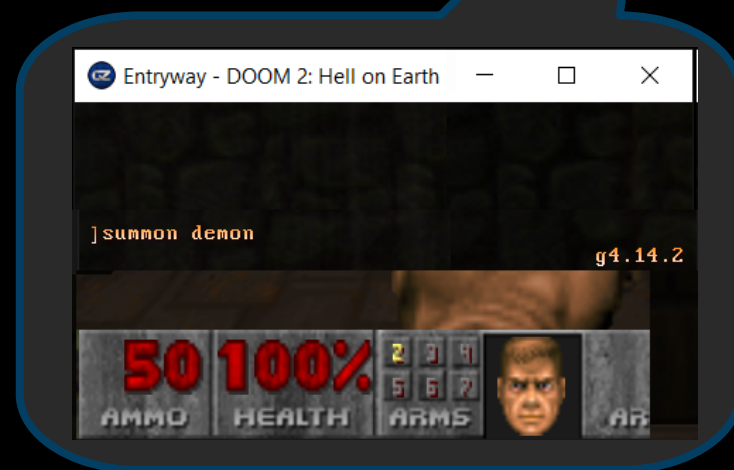
TikFinity-ZDoom Interface Script Getting Started Guide

By Jeremy.Tiggy

This is in no way an exhaustive guide: it's just enough to get started. Please let your curiosity guide you.

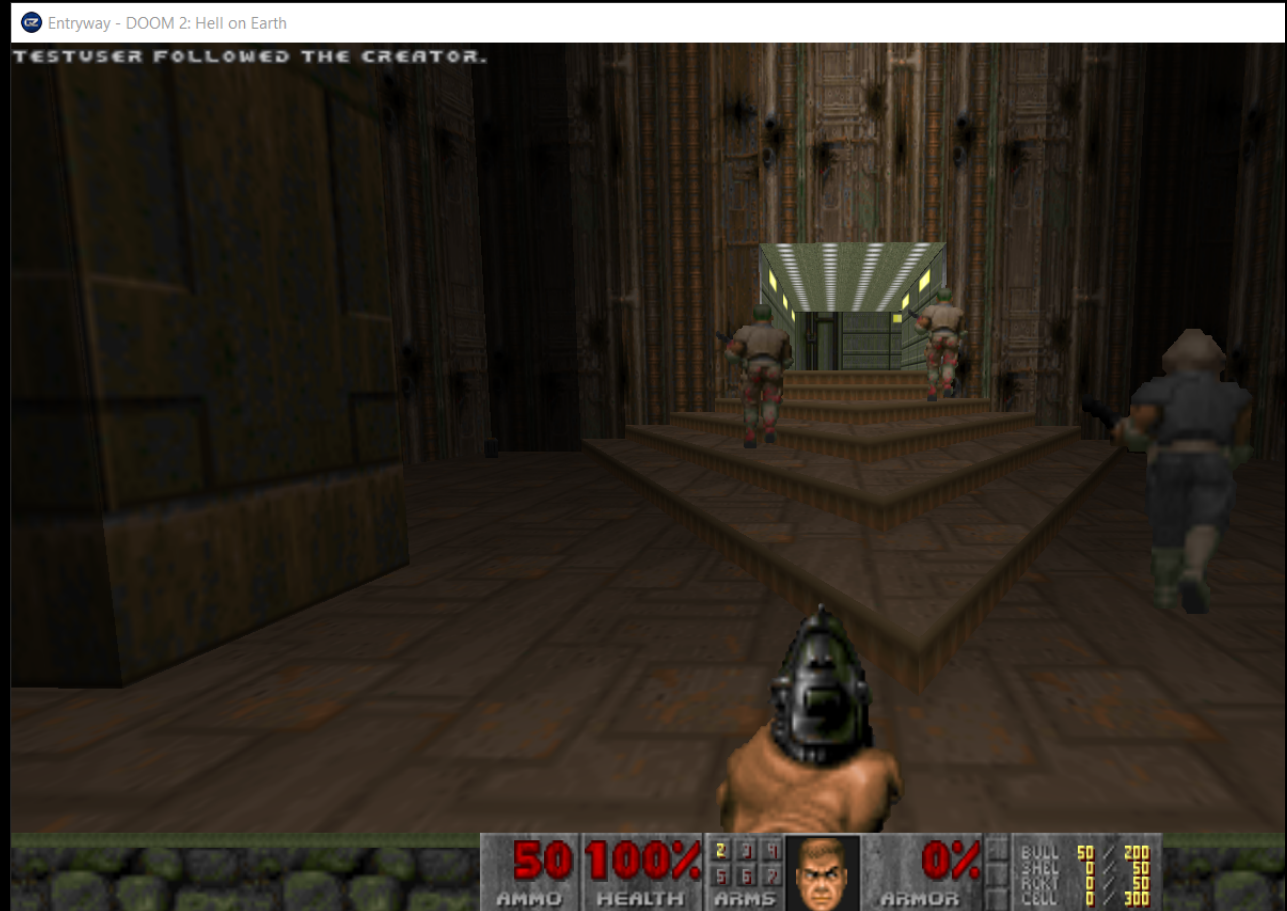


The Script allows TikFinity users to create Event Actions with a Third-Party Category+Action that triggers a console command in ZDoom that executes in the background. The Script assumes that both TikFinity and the Custom ZDoom are running before launch. If either is not running, it will prompt you to start them before continuing.



Demonstration:

1. Follow event gets Action: 'follower Buddy.'
2. Action 'follower Buddy' gets assigned Third-Party Action
3. 'Testuser' follows the creator, activating the event.
4. A Friendly Doom Trooper spawns into existence without interrupting gameplay.
5. A message appears in the top left: 'Testuser followed the creator.'
6. SSG-Buddy kicks ass, 'Testuser' gets credit.



New Action

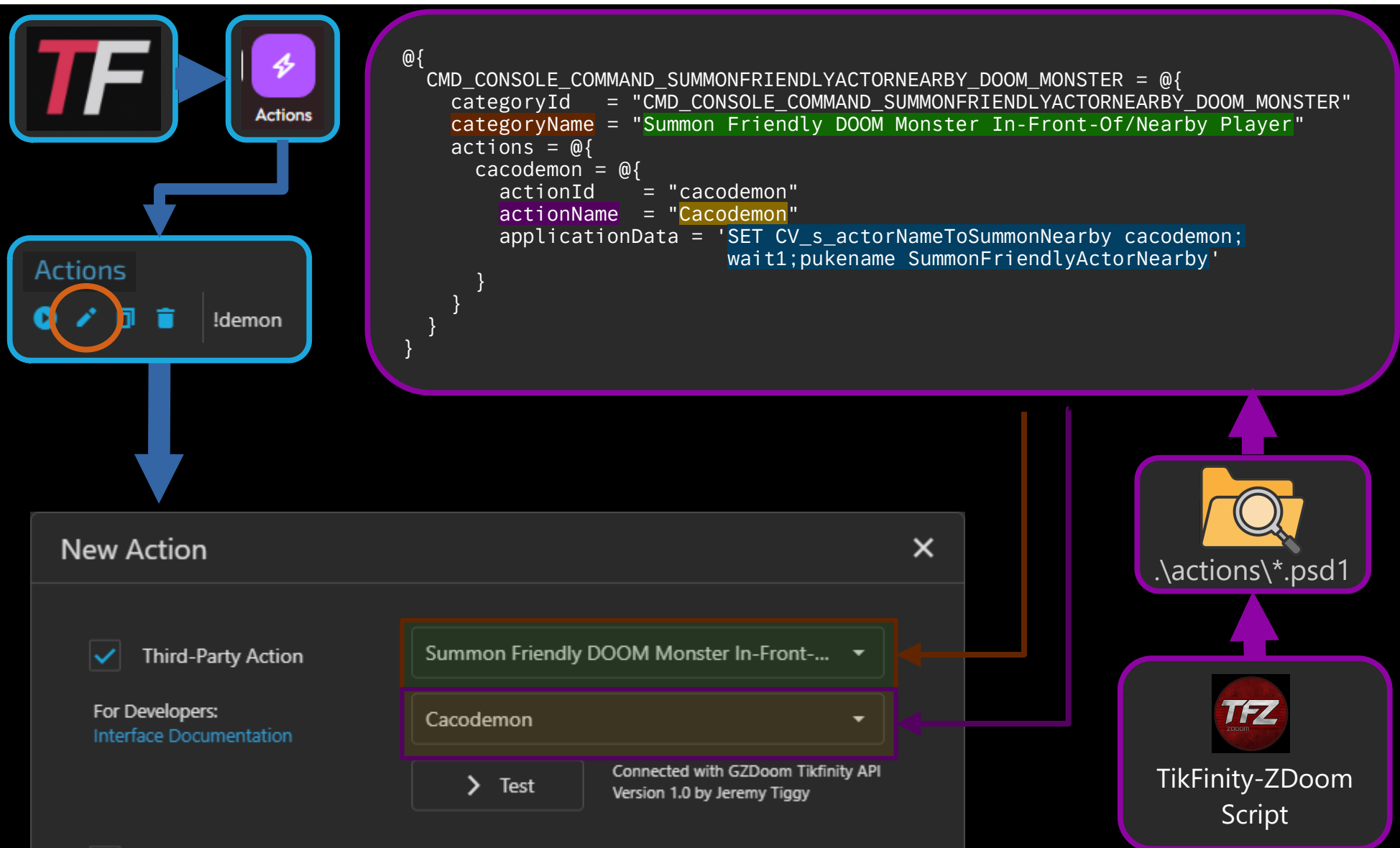
☒ Third-Party Action

Summon Non-Blocking Friendly DOOM Mo...
Doom Trooper w/ Super Shotgun

> Test

Connected with GZDoom Tikfinity API
Version 1.0 by Jeremy Tiggy

Trigger	Action(s)
 Follow	follower Buddy



The Category and Actions list are populated during the Action creation in TikFinity. The Script loads a collection of user data files from the '.actions\' folder in the application directory, then sends over the list to TikFinity. You can edit or make copies of the example file entries to make your own custom Categories and Actions for different DOOM mods. You will need both The Script and the Custom ZDoom running while you're setting up the actions, even if you're not testing.

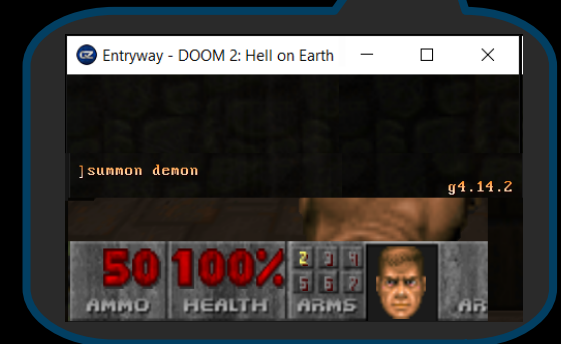
ZDoom API

Pipe:GZD

COMMAND "summon demon"



```
@{
  CMD_SUMMON = @{
    categoryId = "CMD_SUMMON"
    categoryName = "Summon"
    actions = @{
      demon = @{
        actionId = "demon"
        actionName = "Demon"
        applicationData = 'summon demon'
      }
    }
  }
}
```



Events
> !demon

☒ Third-Party Action
Category: Summon
Action: Demon

Port:8832

.\actions*.psd1



TikFinity-ZDoom
Script

The Script has a TCP/IP server listening for Third-Party Action broadcasts from TikFinity. The script looks up the Category and Action pair in its Actions database, and transmits the stored Console Command String to the custom ZDoom application using a special connection called a 'Pipe.' Only the custom ZDoom version 'ExternalPipe' has this functionality.

```
@{
  CMD_SUMMON = @{
    categoryId    = "CMD_SUMMON"
    categoryName  = "Summon"
    actions = @{
      demon = @{
        actionId    = "demon"
        actionName  = "Demon"
        applicationData = 'summon demon'
      }
    }
  }
}
```

Single Console Command

Multiple Console Commands,
Separated with semi-colons, 'wait
1' statements

```
@{
  CMD_CONSOLE_COMMAND_SUMMONFRIENDLYACTORNEARBY_DOOM_MONSTER = @{
    categoryId    = "CMD_CONSOLE_COMMAND_SUMMONFRIENDLYACTORNEARBY_DOOM_MONSTER"
    categoryName  = "Summon Friendly DOOM Monster In-Front-Of/Nearby Player"
    actions = @{
      cacodemon = @{
        actionId    = "cacodemon"
        actionName  = "Cacodemon"
        applicationData = 'SET CV_s_actorNameToSummonNearby cacodemon;
                           wait1;pukename SummonFriendlyActorNearby'
      }
    }
  }
}
```

The Console Commands are stored in the 'applicationData' fields of each action, encased in **single** quotes - Double quotes can be used in the middle, but not the ends. For more complex behaviors, you can string together commands with semicolons. In the bottom example, there is even a 'wait 1' command: this introduces a pause of 1 tick (or 35th of a second) in the sequence, allowing for the previous command time to execute. Doing this between SET and PUKE commands can fix some errors.

Other tools for Console Commands include TikFinity User Data placeholders, indicated by double brackets '{{ }}'. These will be replaced with the data of the User who triggered the event.

Transmitted Data Format from TikFinity

```
{  
  "categoryId": "CMD_CONSOLE_COMMAND_TIKFINITY_EVENT",  
  "actionId": "triggerTypeId1",  
  "context": {  
    "triggerTypeId": 1,  
    "userId": "123456789",  
    "username": "Tesuser"  
  }  
}
```



Script Action Definition

```
@{  
  CMD_CONSOLE_COMMAND_TIKFINITY_EVENT = @{  
    categoryId    = "CMD_CONSOLE_COMMAND_TIKFINITY_EVENT"  
    categoryName  = "TikFinity Trigger"  
  
    actions = @{  
      triggerTypeId1 = @{  
        actionId      = "triggerTypeId1"  
        actionName    = "Share"  
        applicationData = 'echo "{{context.username}}" shares the live.'  
      }  
    }  
  }  
}
```



Tesuser shares the live.

```
]echo "Tesuser shares the live."
```

Final ZDoom
Console Output

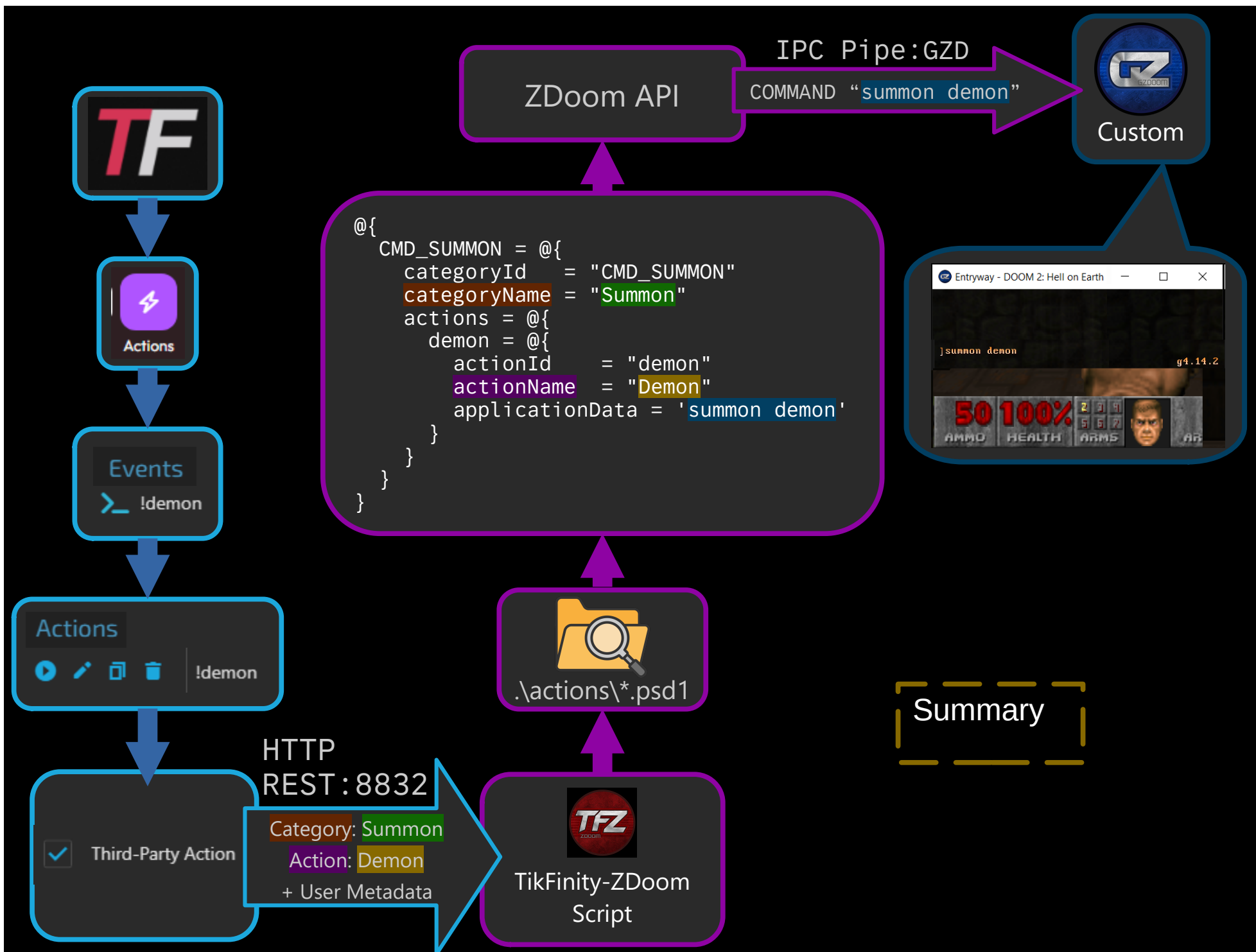


Here's the list of the data that TikFinity supplies to The Script during an event. These can be used for messages (see included Third-Party Action Category 'TikFinity Trigger') or mods.

Trigger Types

Value	Meaning
1	Share
2	Command
3	Gift (minimum coins value trigger)
4	Gift (specific gift trigger)
6	Join
7	Likes (tap event)
9	Follow
10	Subscribe
11	Chat (any message)
12	Emote
13	First User Activity

Placeholder	Description
{{categoryId}}	The category identifier for the action/event.
{{actionId}}	The specific action identifier within the category.
{{context.username}}	The user's account username/login name.
{{context.nickname}}	The user's display name/nickname shown publicly.
{{context.profilePictureUrl}}	URL to the user's profile picture/avatar image.
{{context.giftId}}	Numeric identifier for the gift involved in the event.
{{context.giftName}}	Human-readable name of the gift involved in the event.
{{context.coins}}	Coin value associated with the gift or event.
{{context.repeatCount}}	Number of repeated gifts/actions in the current streak or batch.
{{context.likeCount}}	Number of likes/taps involved in the current like event.
{{context.totalLikeCount}}	User's cumulative total likes during the stream/session.
{{context.subMonth}}	Subscription month count or subscription duration.
{{context.emoteId}}	Identifier for the emote used in the event.
{{context.comment}}	The chat message/comment text submitted by the user.
{{context.triggerTypeId}}	Event Type that triggered the action



Normal Script Startup - Most Common, Auto

```
[Enter Command (debug|normal or press enter to skip)]:>
[Startup]: Starting communications...
[GZDoom_REST_Startup] Starting up Pipe Client and HTTP Listener
[NamedPipe_Client_Startup]: Pipe: GZD
[NamedPipe_Client_Startup]: Process: GZDoom - RUNNING
[NamedPipe_Client_ConnectToServer]: Connecting to pipe: GZD
[NamedPipe_Client_Startup]: Connected to Pipe GZD successfully!
[NamedPipe_Client_Startup]: Connected: True
[REST_Server_Startup]: Port: 8832
[REST_Server_Startup]: URI: http://127.0.0.1:8832/
[REST_Server_Startup]: Process: Tikfinity - RUNNING
[REST_Server_Startup]: appInfo: GZDoom Tikfinity API version 1.0 by Jeremy Tiggy
[REST_Server_Startup]: Listener started successfully!
[Startup]: Client Connected To Named Pipe Server
[Startup]: HTTP REST Response Server is running.
[Startup]: The Pipe is connected and the Server is running.

[Startup]: Starting main loop...
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...
```

Normal Script Startup, Pipe

```
[Startup]: Would you like to start up in 'debug' mode or 'normal'?  
Type 'debug' to walk-thru each sub-system as it starts up.  
This is helpful to be able to get support.  
Type 'normal' or just hit Enter to continue in Automatic Mode.  
[Enter Command (debug|normal or press enter to skip)]:> _
```

Normally, press enter (or type 'normal') to continue execution.
The first subsystem to startup is the special connection between The Script and Zdoom: the Pipe.

```
[Startup]: Starting communications...  
[GZDoom_REST_Startup] Starting up Pipe Client and HTTP Listener  
[NamedPipe_Client_Startup]: Pipe: GZD  
[NamedPipe_Client_Startup]: Process: GZDoom - NOT RUNNING  
[NamedPipe_Client_Startup]: Connected: False  
[NamedPipe_Client_Startup]: Would you like to attempt to OPEN the named pipe GZD, change the TARGET pipe  
name before opening, or work OFFLINE?  
[NamedPipe_Client_Startup] (open|target=GZD@GZDoom|offline|exit)> _
```

If the special version of ZDoom is not running when The Script starts, you will be notified. If you then start the custom ZDoom, go back to the window and type **open**. If you are running a different custom version of Doom, you can change the **target** here. Running **offline** is suitable for setting up Actions in TikFinity, but not for testing those actions.

```
[NamedPipe_Client_Startup]: Pipe: GZD  
[NamedPipe_Client_Startup]: Process: GZDoom - RUNNING  
[NamedPipe_Client_ConnectToServer]: Connecting to pipe: GZD  
[NamedPipe_Client_Startup]: Connected to Pipe GZD successfully!  
[NamedPipe_Client_Startup]: Connected: True
```

Normal Script Startup, REST Server

```
[REST_Server_Startup]: Port: 8832
[REST_Server_Startup]: URI: http://127.0.0.1:8832/
[REST_Server_Startup]: Process: Tikfinity - NOT RUNNING
[REST_Server_Startup]: appInfo: GZDoom Tikfinity API version 1.0 by Jeremy Tiggy
[REST_Server_Startup]: Since Tikfinity is not running, starting the server may not provide connectivity.
[REST_Server_Startup]: You can still continue in offline mode and enter the Event Data manually.
[REST_Server_Startup]: Would you like to 'start' the server, or work 'offline'?
[Enter Command (start|offline|exit)]:>
```

If TikFinity isn't running when The Script starts up, The Script will pause. Open TikFinity before typing **start**. Working **offline** is helpful when you'd like to **simulate** data for developing mods and scripts.

```
[Main Loop]: Enter Command (exit|listen|simulate|close)> simulate
[Simulate]: Incoming Third Party Action Data

[Edit-ObjectInteractive] categoryId
  Current : categoryId (String)
  Enter new value (blank = keep): CMD_CONSOLE_COMMAND_TIKFINITY_EVENT

[Edit-ObjectInteractive] actionId
  Current : actionId (String)
  Enter new value (blank = keep): triggerTypeId9

[Edit-ObjectInteractive] context.userID
  Current : userID (String)
  Enter new value (blank = keep):

[Edit-ObjectInteractive] context.username
  Current : username (String)
  Enter new value (blank = keep): Tesuser
```

```
MAP01 - Entryway
TikfinityEventTriggers - Started
TikfinityEventData - Started
Tesuser has followed JeremyTiggy.
Tesuser followed the creator.
]
```

Normal Script Startup, REST Server

```
[REST_Server_Startup]: Port: 8832
[REST_Server_Startup]: URI: http://127.0.0.1:8832/
[REST_Server_Startup]: Process: Tikfinity - NOT RUNNING
[REST_Server_Startup]: appInfo: GZDoom Tikfinity API version 1.0 by Jeremy Tiggy
[REST_Server_Startup]: Since Tikfinity is not running, starting the server may not provide connectivity.
[REST_Server_Startup]: You can still continue in offline mode and enter the Event Data manually.
[REST_Server_Startup]: Would you like to 'start' the server, or work 'offline'?
[Enter Command (start|offline|exit)]:> start
[REST_Server_Startup]: Listener started successfully!
[Startup]: Client Connected To Named Pipe Server
[Startup]: HTTP REST Response Server is running.
[Startup]: The Pipe is connected and the Server is running.

[Startup]: Starting main loop...
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...
```

The REST Server can be activated during start-up by typing `start`; but until TikFinity is running, The Script will be stuck waiting for data that will not be coming.

Normal Script Startup, REST Server

```
[Startup]: Starting main loop...
[Main Loop]: To quit, type 'exit'.
[Main Loop]: Type 'listen' to start the REST HTTP Server on http://127.0.0.1:8832/
[Main Loop]: Type 'simulate' to manually put together an event with data.
[Main Loop]: Type 'close' to end the Named Pipe connection 'GZD @ GZDoom'
[Main Loop]: Enter Command (exit|listen|simulate|close)> listen
[REST_Server_Startup]: Port: 8832
[REST_Server_Startup]: URI: http://127.0.0.1:8832/
[REST_Server_Startup]: Process: Tikfinity - RUNNING
[REST_Server_Startup]: appInfo: GZDoom Tikfinity API version 1.0 by Jeremy Tiggy
[REST_Server_Startup]: Listener started successfully!
[Main Loop]: To quit, type 'exit'.
[Main Loop]: Type 'block' to wait for a HTTP request.
[Main Loop]: Type 'simulate' to manually put together an event with data.
[Main Loop]: Type 'close' to end the Named Pipe connection 'GZD @ GZDoom'
[Main Loop]: Type 'auto' to leave manual control and send REST Actions continuously to GZDoom.
[Main Loop]: Enter Command (exit|block|simulate|close|auto)> _
```

The REST Server can be activated after startup from the Main Menu by typing **listen**. If started this way, you can use **auto** to resume normal continuous operation. Use **block** to listen and respond to a single message, which is helpful when modifying the core script or troubleshooting; most REST communication has several steps, so to finish the request, you may need to **block** several times.

```
[Main Loop]: Enter Command (exit|block|simulate|close|auto)> block
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...
[Main Loop]: To quit, type 'exit'.
[Main Loop]: Type 'block' to wait for a HTTP request.
[Main Loop]: Type 'simulate' to manually put together an event with data.
[Main Loop]: Type 'close' to end the Named Pipe connection 'GZD @ GZDoom'
[Main Loop]: Type 'auto' to leave manual control and send REST Actions continuously to GZDoom.
[Main Loop]: Enter Command (exit|block|simulate|close|auto)> block
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...
[REST_API_Handle-Request]: INFO - App Info Request serviced
```

Script Debug Mode - all

```
[Enter Command (debug|normal or press enter to skip)]:> debug
[Startup]: Type 'select' to pick which systems to debug.
          Type 'all' or press Enter to print out information on everything.
[Enter Command (select|all or press enter to skip)]:> _
```

When you first pick **debug** from the script start up prompt, you can either **select** what systems to debug, or show information from **all** script subsystems and manually start them up one-by-one. (which is the most common debugging method.)

```
[Enter Command (select|all or press enter to skip)]:> all
[Startup]: Starting communications...
[GZDoom_REST_Startup] Starting up Pipe Client and HTTP Listener
[NamedPipe_Client_Startup]: Pipe: GZD
[NamedPipe_Client_Startup]: Process: GZDoom - RUNNING
[NamedPipe_Client_ConnectToServer]: Connecting to pipe: GZD
[NamedPipe_Client_Startup]: Connected to Pipe GZD successfully!
[NamedPipe_Client_Startup]: Connected: True
[REST_Server_Startup]: Port: 8832
[REST_Server_Startup]: URI: http://127.0.0.1:8832/
[REST_Server_Startup]: Process: Tikfinity - RUNNING
[REST_Server_Startup]: appInfo: GZDoom Tikfinity API version 1.0 by Jeremy Tiggy
[REST_Server_Startup]: Since Tikfinity is running, it's recommended to start the server to enable connectivity.
[REST_Server_Startup]: Would you like to 'start' the server, or work 'offline'?
[Enter Command (start|offline|exit)]:> start
[REST_Server_Startup]: Listener started successfully!
[Startup]: Client Connected To Named Pipe Server
[Startup]: HTTP REST Response Server is running.
[Startup]: The Pipe is connected and the Server is running.
[Startup]: Press Enter to continue in Automatic Mode
[Startup]: Type 'manual' to continue in Manual Mode (you can always enter Auto later)
[Startup]: (manual|leave blank and hit enter for auto) >

[Startup]: Starting main loop...
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...
```

Script Debug Mode - Select, Pipe

```
[Enter Command (select|all or press enter to skip)]:> select
```

```
[Startup]: Enable Pipe Debugging Messages?
```

```
[Enter Command (yes|no)]:> yes
```

```
[Startup]: Enable HTTP Server Debugging Messages?
```

```
[Enter Command (yes|no)]:> yes
```

```
[Startup]: Enable REST API Debugging Messages?
```

```
[Enter Command (yes|no)]:> yes
```

```
[Startup]: Enable GZDoom API Debugging Messages?
```

```
[Enter Command (yes|no)]:> yes
```

When you **select** what systems to debug, you can focus on the Pipe, HTTP Server, REST API, and ZDoom API systems.

Pipe Debugging shows the details of the communication between The Script and the custom ZDoom's special Pipe communication link. This includes the API functions - GET, SET, and COMMAND – and any errors surrounding the integrity of the Pipe.

```
[GZDoom_PipeAPI_CVAR_SET]: [Raw function input] CVAR: CV_s_TF_categoryId
[GZDoom_PipeAPI_CVAR_SET]: [Raw function input] Value: CMD_CONSOLE_COMMAND_SUMMONMBF_DOOM_MONSTER
[GZDoom_PipeAPI_CVAR_SET]: [Function Input as String] CVAR: CV_s_TF_categoryId
[GZDoom_PipeAPI_CVAR_SET]: [Function Input As String] Value: CMD_CONSOLE_COMMAND_SUMMONMBF_DOOM_MONSTER
[GZDoom_PipeAPI_CVAR_SET]: Request String: SET CV_s_TF_categoryId CMD_CONSOLE_COMMAND_SUMMONMBF_DOOM_MONSTER
[GZDoom_PipeAPI_CVAR_SET]: [Raw] Response String: "CV_s_TF_categoryId" is "CMD_CONSOLE_COMMAND_SUMMONMBF_DOOM_MONSTER"
[GZDoom_PipeAPI_CVAR_SET]: Response: GZDoom reports the value of remote CVAR 'CV_s_TF_categoryId' has changed to a requested value 'CMD_CONSOLE_COMMAND_SUMMONMBF_DOOM_MONSTER'.
[GZDoom_PipeAPI_CVAR_SET]: Response Code: 1
[GZDoom_PipeAPI_CVAR_SET]: Response CVAR Name: CV_s_TF_categoryId
[GZDoom_PipeAPI_CVAR_SET]: Response CVAR Value: CMD_CONSOLE_COMMAND_SUMMONMBF_DOOM_MONSTER
[GZDoom_PipeAPI_CVAR_SET]: Response CVAR Value: CMD_CONSOLE_COMMAND_SUMMONMBF_DOOM_MONSTER
[GZDoom_PipeAPI_CMD_CVAR_Update_Local]: Remote CVAR 'CV_s_TF_categoryId' is now 'CMD_CONSOLE_COMMAND_SUMMONMBF_DOOM_MONSTER'
```

Script Debug Mode - HTTP Server

```
[Startup]: Starting main loop...  
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...  
[REST_Server_Wait-For-Request]: DEBUG - Request Detected  
[REST_Server_Add-CORSHeaders]: DEBUG - Adding CORS headers  
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...  
[REST_Server_Wait-For-Request]: DEBUG - Request Detected  
[REST_Server_Add-CORSHeaders]: DEBUG - Adding CORS headers  
[REST_API_Handle-Request]: INFO - Execute Action Request serviced  
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...
```

The HTTP Server debugging covers the handling of POST requests.

Script Debug Mode - REST API

The REST API debugging covers the formatting of JSON data received and sent for categories and actions.

```
[Startup]: Starting main loop...  
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...  
[REST_API_Handle-Request]: DEBUG - Processing request - Method: OPTIONS, URL: http://127.0.0.1:8832/api/features/categories?  
[REST_API_Handle-Request]: DEBUG - Handling OPTIONS (preflight) request  
[REST_API_Handle-Request]: INFO - OPTIONS request serviced  
[REST_Server_Wait-For-Request]: BLOCKING - Waiting for incoming HTTP requests...  
[REST_API_Handle-Request]: DEBUG - Processing request - Method: GET, URL: http://127.0.0.1:8832/api/features/categories?  
  
[REST_API_Handle-Request]: DEBUG - Action Categories Info Requested  
Count: 14  
Length: 14  
LongLength: 14  
Rank: 1  
SyncRoot:  
    categoryId: CMD_CONSOLE_COMMAND_PUKE  
    categoryName: Execute Script by Number
```

Included Mod: EventDataDefinitions (Used To Make Your Own Mods)

The Script includes a .PK3 file, EventDataDefinitions, that is automatically loaded with the Custom ZDoom. It's purpose is to define the TikFinity Event & User Metadata variables, and to acknowledge/reset the global "New Data From TikFinity Has Arrived" information variable for other mods to use in their logic for handling that data. The Script sends over all available received User, Event, and Action Data to ZDoom for mod usage.

CVAR	TikFinity Data Description
CV_s_TF_categoryId	The category identifier for the action/event.
CV_s_TF_categoryName	
CV_s_TF_actionId	The specific action identifier within the category.
CV_s_TF_actionId	
CV_s_TF_username	The user's account username/login name.
CV_s_TF_nickname	The user's display name/nickname shown publicly.
CV_s_TF_profilePictureUrl	URL to the user's profile picture/avatar image.
CV_n_TF_giftId	Numeric identifier for the gift involved in the event.
CV_s_TF_giftName	Human-readable name of the gift involved in the event.
CV_n_TF_coins	Coin value associated with the gift or event.
CV_n_TF_repeatCount	Number of repeated gifts/actions in the current streak
CV_n_TF_likeCount	Number of likes/taps involved in the current like event.
CV_n_TF_totalLikeCount	User's cumulative total likes during the stream/session.
CV_n_TF_subMonth	Subscription month count or subscription duration.
CV_n_TF_emoteId	Identifier for the emote used in the event.
CV_s_TF_comment	The chat message/comment text submitted by the user.
CV_n_TF_triggerTypeId	Event Type that triggered the action

ACTIONS & EVENTS DATA FROM TIKFINITY

- Action -
Category ID categoryId
Category Name categoryName
Action ID actionId
Action Name Action Name

- User -
User ID userID
User Name username
User Nickname Nick Name
User Profile Picture https://

- Event -
Trigger Id 987.000
Trigger Type Unknown

- Gift -
Gift ID 0.000
Gift Name Rose
Coins 999.000
Repeat Count 0.000

- Likes -
Like Count 0.000
Total Like Count 0.000

- Subscription -
Number of Months Subscribed 0.000
- Emote -
Emote ID 0.000

- Message or Command -
Chat chat message
- Tikfinity -
Tikfinity User ID 654.000
Tikfinity Username tikfinityUsername

The Actions & Events Data From TikFinity menu shows the last received data sent by TikFinity. The Data Update Trigger can be toggled from Off to On to refresh any triggers in any mods.

- Data Update Trigger -
Data Updated Signal from GZDoom-REST API Off
Ticks Elapsed until reset of Data Ready signal 10.000
Debugging Messages Off

The Ticks Elapsed is the delay before the mod automatically resets the signal - Too low, and mods might not catch the update or share the CPU; too high, events might be missed. Mods should therefore not have any Delay commands totaling longer than this value.

Included Mod: EventDataDisplay (Good Example of a Simple Mod)

EVENT DATA FROM TIKFINITY

EventDataDisplay.pk3 adds a menu that is much like the other TikFinity menu, with one important functional difference: The Enable Display Messages for Events with Trigger. When On, Messages will appear for each TikFinity Event Trigger – for example, if someone were to send a gift, their name and the gift would show up as a message.

Trigger Id 987.000

Trigger Type Unknown

– Gift –

Gift ID 0.000

Gift Name Rose

Coins 0.000

Repeat Count 0.000

– Likes –

Like Count 0.000

Total Like Count 1.000

– Subscription –

Number of Months Subscribed 0.000

– Emote –

Emote ID 1.000

– Message or Command –

Chat chat message

– Data Update Trigger –

Data Updated Signal from GZDoom-REST API Off

Enable Display Messages for Events with Trigger On

Enable Debugging Messages Off

Included Mod: eventDataDisplay (Good Example of a Simple Mod)

Internal to the mod is logic to automatically show TifFinity event messages with actions, so that even if an action doesn't include instructions to echo a message, it will pop up anyways. The most important structure is the readNewData rising edge detection and subsequent read of CV_s_TF_username.

```
script "TikfinityEventTriggersOpen" OPEN NET {
    Log(s:"TikfinityEventTriggers - Started");
    bool newData_ONS = false;
    while (1 > 0) {
        bool debug = GetCVAR("CV_b_TikfinityEventTriggers_debug");
        bool newData = GetCVAR("CV_b_API_dataReady");
        bool readNewData = newData && !newData_ONS;
        bool printMessages = GetCVAR("CV_b_API_showTFmessages");
        newData_ONS = newData;
        if (readNewData) {
            int n_triggerTypeId = GetCVAR("CV_n_TF_triggerTypeId");
            if (printMessages) {

                str s_triggerType = "unknown";
                bool makeChatSound = true;
                switch (n_triggerTypeId) {
                    case 1:
                        s_triggerType = "Shared";
                        Log(s:GetCVARString("CV_s_TF_username"),s:" shared the live.");
                        break;
                        ...
                    default:
                        s_triggerType = "Not Known";
                        makeChatSound = false;
                        break;
                } //end switch
                if (makeChatSound) AmbientSound("misc/chat", 127);
            } // end if (printMessages)
        } // end if (newdata)
        delay(1);
    } // end while
} // end script
```

Included Mods: SummonNearby (Prevents Actors Stuck In Walls)

SUMMON NEARBY ACTOR OPTIONS

Usage Example - Summon zombieman:

set CV_s_actorNameToSummonNearby zombieman;pukename SummonActorNearby

Usage Example - Summon Caco-Buddy:

set CV_s_actorNameToSummonNearby cacodemon;pukename SummonFriendlyActorNearby

Settings:

Distance from Player ◀ 160.0

[Last] Actor Name empty

Commands:

- > Test Spawn Actor <
- > Test Spawn Friendly Actor <
- > Test Spawn Explosive Barrel <
- > Test Spawn Cyber-Buddy <

The Summon Nearby Actor Options menu sets the distance that actors spawn using the SummonActorNearby and SummonFriendlyActorNearby functions. These functions are used by the included 'Summon X In-Front-Of/Nearby Player' actions. In the example, barrels were summoned at 5 (Minimum), 250 (Middle), and 500 (Max).



The External Pipe Options menu shows the status of the pipe and data logging. If The Script is having any problems connecting to ZDoom, use the 'Toggle Pipe Enable' to disable the Pipe Logic, then re-Enable it. This should fix most problems. The Cycle Timers dictate the internal behavior of the Pipe communications; these numbers ensure that comms don't eat-up too much processor overhead.

Included Mods: ExternalPipeMenu

EXTERNAL PIPE OPTIONS

--- Pipe Status (Read-Only) ---

Pipe Logic	Enabled
Server Connections	Open
Client Connections	Disconnected
Log Pipe Operations	Disabled

Pipe Operation Log Destination	Logfile Only
+Logfile present in command line	Present
Logfile Name provided in command line [Read-Only]	logfile.txt

> Print Logfile Name To Console <

Current Latency (milliseconds) [Read-Only]	1000.000
--	----------

Last Data From Client

Last Data To Client

> Print Client Data To Console <

!> Toggle Pipe Enable <!

--- Cycle Timers (Ticks) ---

Pipe Disconnected, Attempt to Connect	35.000
---------------------------------------	--------

Pipe Connected, Fresh Data	3.000
----------------------------	-------

Pipe Connected, Idle	7.000
----------------------	-------

Idle Threshold (How many cycles without new data)	10.000
---	--------

Included Program: DoomRunner

Included with the installation package is a copy of DoomRunner: a launcher and mod manager for ZDoom. It comes preconfigured with the custom ZDoom program, and looks to the local Maps directory for the map listing. For more information on how to get started with DoomRunner, please check out youtube. Please use this to launch the custom ZDoom version as a part of startup. It will keep to keep your sanity intact.

<https://www.youtube.com/watch?v=aujZILMrEQU>

<https://github.com/Youda008/DoomRunner>

